



# *DataHarvest*

## **RAPPORT TECHNIQUE**

---

Gurvan Godin, Maxime Danino  
Master Dev, Data & IA, 4e année  
IPSSI Nice

31 juillet 2026

*Framework de scraping modulaire, projet final Web Scraping*

# 1. Introduction et périmètre

---

## Le problème que DataHarvest essaie de résoudre

Pendant ce module, nous avons appris à récupérer des données sur le web avec plusieurs outils : [requests](#) combiné à [BeautifulSoup4](#) pour parser du HTML simple, Selenium pour piloter un vrai navigateur quand une page a besoin de JavaScript, et Scrapy pour des projets de scraping plus ambitieux. Chacun de ces outils fonctionne très bien dans son contexte, mais aucun n'est vraiment pensé pour un usage à mi-chemin entre les deux : trop léger pour justifier un vrai projet Scrapy, mais avec assez de sites différents à traiter pour qu'un script écrit à la main pour chacun devienne vite pénible à maintenir.

C'est ce problème que nous avons voulu résoudre avec DataHarvest. L'idée de départ est simple : plutôt que d'écrire un script Python différent pour chaque site, on décrit le site dans un fichier de configuration (quelle page visiter, quelles informations récupérer, comment gérer la pagination), et un seul programme se charge de tout le reste. Ajouter un sixième site à traiter ne demande donc pas de toucher au code, seulement d'écrire un nouveau fichier de configuration.

Ce fonctionnement est proche de ce que propose déjà Scrapy, mais en beaucoup plus simple et plus petit. L'objectif du projet était moins de rivaliser avec Scrapy que de comprendre, en le construisant nous-mêmes, ce que ce genre d'outil doit résoudre en interne : comment séparer proprement le téléchargement, l'extraction et la validation des données, et pourquoi ce découpage a de l'importance dans un vrai projet.

## Ce que DataHarvest ne fait pas

Nous avons volontairement limité le projet à ce qui nous semblait raisonnable de réaliser en cinq heures de développement à deux. DataHarvest ne sait scraper que des pages HTML classiques, obtenues par une simple requête HTTP. Il ne gère pas les sites qui n'affichent leur contenu qu'après exécution de JavaScript, ni les formulaires de connexion, ni le défilement infini. Il ne devine pas non plus tout seul quelles informations récupérer sur une page : c'est à la personne qui écrit la configuration d'aller regarder le code source du site, avec les outils de développement du navigateur, pour trouver les bons éléments à cibler. Enfin, DataHarvest reste un simple programme en ligne de commande. Il ne se lance pas tout seul à intervalles réguliers et n'a pas d'interface graphique.

Ce ne sont pas des oublis, mais des choix assumés dès le départ, pour rester sur un projet réalisable dans le temps imparti plutôt que de viser un outil aussi complet qu'un vrai framework de production.

## Les sites que nous avons choisis

Le sujet demandait de scraper au moins cinq sites en couvrant au moins deux niveaux de difficulté parmi les quatre proposés. Nous en avons choisi cinq, répartis sur trois niveaux différents, ce qui nous semblait être un bon compromis entre diversité des structures HTML et faisabilité dans le temps imparti.

Pour le niveau 1, le plus simple, nous avons pris [books.toscrape.com](#) et [quotes.toscrape.com](#). Ce sont deux sites d'entraînement pensés spécifiquement pour apprendre le scraping, avec une structure HTML stable et sans aucun risque de bloquer les robots. Ils nous ont surtout servi à vérifier

rapidement que les bases de DataHarvest (extraction des données, gestion de la pagination) fonctionnaient correctement, avant de nous attaquer à des sites plus réalistes et donc plus imprévisibles.

Pour le niveau 2, nous avons choisi deux sites de données publiques francophones : [fr.wikipedia.org](http://fr.wikipedia.org) et [data.gouv.fr](http://data.gouv.fr). Wikipédia nous a permis de tester DataHarvest sur un tableau structuré, une liste de pays classés par population, tandis que data.gouv.fr propose un vrai catalogue de jeux de données publiques, avec ses propres organisations et ses propres conventions d'affichage.

Enfin, pour le niveau 3, nous avons pris [blogdumoderateur.com](http://blogdumoderateur.com), un site d'actualité du numérique avec une pagination classique et des articles classés par catégorie. C'est le site qui se rapproche le plus d'un cas d'usage réel de veille ou de collecte d'actualités, et celui qui nous a d'ailleurs donné le plus de fil à retordre (voir section 5).

Cette diversité de sites, boutique fictive, recueil de citations, encyclopédie, catalogue de données publiques et blog d'actualité, nous permet de montrer que DataHarvest fonctionne sur des structures de page très différentes sans jamais changer une ligne de code : seule la configuration change d'un site à l'autre.

## 2. Architecture et choix de conception

---

### Pourquoi avoir découpé le projet en plusieurs briques plutôt que d'écrire un seul gros script

Un script de scraping écrit rapidement mélange en général tout dans la même fonction : on télécharge la page, on en extrait les informations, on vérifie qu'elles ont du sens, puis on les enregistre, le tout à la suite. Ça fonctionne très bien pour un script rapide sur un seul site, mais ça devient vite difficile à faire évoluer dès qu'on veut réutiliser une partie du code ailleurs, ou simplement tester un morceau du programme sans dépendre du réseau.

Nous avons donc préféré séparer DataHarvest en plusieurs briques indépendantes, chacune avec un rôle bien précis : une brique qui lit la configuration, une qui télécharge les pages, une qui en extrait les données, une qui vérifie que ces données sont correctes, et une dernière qui les enregistre. Un dernier morceau, l'Orchestrator, fait le lien entre toutes les autres et pilote le déroulement complet d'un scraping.

Ce découpage nous a apporté trois avantages très concrets pendant le développement. D'abord, chaque brique peut être testée toute seule, sans dépendre des autres ni d'une vraie connexion internet. Par exemple, nos tests sur le téléchargement des pages utilisent une fausse réponse HTTP préparée à l'avance plutôt qu'une vraie requête, ce qui les rend rapides et fiables même sans connexion. Ensuite, ajouter une nouvelle fonctionnalité, un nouveau format d'enregistrement par exemple, ne demande de toucher qu'un seul fichier, sans prendre le risque de casser le reste du programme au passage. Enfin, le code est plus facile à comprendre pour quelqu'un qui le découvre pour la première fois, puisque chaque fichier ne fait qu'une seule chose et porte un nom qui explique clairement son rôle.

### Pourquoi le squelette d'extraction impose certaines règles

Dans DataHarvest, la brique qui extrait les données d'une page doit toujours proposer deux choses : savoir extraire les informations d'une page, et savoir déterminer s'il existe une page suivante. Plutôt que de simplement noter cette règle dans un commentaire, ce qu'on pourrait facilement oublier de respecter en cours de développement, nous avons utilisé une fonctionnalité de Python appelée classe abstraite, qui empêche techniquement de créer une nouvelle brique d'extraction si elle ne respecte pas ces deux règles.

Concrètement, si quelqu'un oublie d'écrire une de ces deux fonctions en créant un nouveau type d'extraction, le programme refuse de démarrer immédiatement, avec un message d'erreur clair, plutôt que de planter beaucoup plus tard, en plein milieu d'un scraping, pour une raison difficile à retrouver. C'est une petite sécurité, mais qui évite une catégorie entière d'erreurs d'inattention.

### Pourquoi un système en chaîne pour les requêtes HTTP

Le téléchargement d'une page a souvent besoin de traitements annexes : afficher un message dans la console pour suivre l'avancement, réessayer automatiquement en cas d'erreur du serveur, ou, plus tard, limiter le nombre de requêtes envoyées à un même site pour ne pas le surcharger. Si on avait codé tout ça directement dans la fonction qui télécharge les pages, cette fonction serait vite devenue longue et difficile à modifier sans risquer de casser autre chose au passage.

Nous avons préféré un système où chaque traitement annexe est écrit dans sa propre petite classe, toutes empilées les unes après les autres autour du téléchargement proprement dit. C'est le même principe que celui utilisé par des frameworks web très répandus comme Django ou Flask pour traiter leurs propres requêtes. Chaque brique de ce type peut être testée, modifiée ou retirée sans toucher aux autres, et sans jamais modifier le code de téléchargement lui-même.

## **Pourquoi un fichier de configuration plutôt que des options en ligne de commande**

Décrire un site à scraper demande plusieurs informations liées entre elles : l'adresse de départ, la façon dont la pagination fonctionne, les différents éléments à cibler pour chaque information à récupérer, et les réglages du téléchargement. Faire passer tout ça par des options de ligne de commande aurait donné des commandes à rallonge, difficiles à relire et à corriger en cas d'erreur.

Un fichier YAML se prête beaucoup mieux à ce genre d'information organisée : il se lit facilement même après plusieurs semaines, se conserve dans le dépôt Git comme n'importe quel autre fichier, et documente lui-même le site qu'il décrit. Les options en ligne de commande restent tout de même utiles pour ce qui concerne le déroulement d'une exécution en particulier, comme choisir quel fichier de configuration charger, ou activer un mode d'essai qui n'enregistre rien.

## **Pourquoi construire les briques depuis l'extérieur plutôt que les appeler directement**

Chaque brique de DataHarvest reçoit ce dont elle a besoin au moment de sa création, plutôt que d'aller chercher elle-même ses dépendances dans son propre code. La brique qui télécharge les pages, par exemple, reçoit sa configuration et sa liste de traitements annexes de l'extérieur, plutôt que de les importer elle-même en dur.

Cette façon de faire permet de remplacer facilement un morceau du programme par une version simplifiée lors des tests automatisés. C'est exactement ce que nous avons fait dans nos propres tests : pour vérifier le comportement en cas d'erreur réseau, nous avons remplacé la vraie connexion internet par une fausse réponse préparée à l'avance, sans avoir besoin de modifier le code de la brique elle-même pour l'occasion.

### 3. Comparaison avec Scrapy

---

#### Ce que Scrapy propose et que nous n'avons pas eu le temps de faire

Scrapy est un framework de scraping utilisé en production dans de nombreuses entreprises, et il propose beaucoup plus de fonctionnalités que DataHarvest. Nous en avons identifié cinq qui nous semblaient particulièrement représentatives de l'écart entre les deux outils.

La première est la capacité de Scrapy à envoyer des centaines de requêtes en même temps, grâce à un système de programmation asynchrone. DataHarvest, de son côté, télécharge les pages une par une, ce qui le rend beaucoup plus lent sur un gros volume de pages. La deuxième est un système de cache qui permet à Scrapy d'éviter de retélécharger une page déjà récupérée lors d'une exécution précédente, ce que DataHarvest ne fait pas encore. La troisième est un ajustement automatique du délai entre les requêtes selon la vitesse de réponse du site visité, alors que DataHarvest se contente d'un délai fixe défini dans la configuration. La quatrième est la possibilité, avec Scrapy, de suivre automatiquement des liens selon des règles définies à l'avance, sans avoir à écrire soi-même la logique de pagination pour chaque site, ce que DataHarvest impose de faire manuellement dans chaque configuration. La dernière est un système d'export capable de produire directement plusieurs formats de fichiers en une seule commande, y compris pour de très gros volumes de données, alors que DataHarvest gère cela de façon plus artisanale, avec une logique un peu différente pour chaque format de sortie.

Aucune de ces fonctionnalités n'est impossible à réaliser avec plus de temps. Chacune représentait, à elle seule, un travail bien plus important que ce que permettait un projet de cinq heures à deux personnes.

#### Dans quels cas préférer DataHarvest à Scrapy malgré tout

Il existe quand même des situations où DataHarvest peut être préférable à Scrapy. La première est un besoin ponctuel sur un site simple : monter un projet Scrapy complet, avec tous ses fichiers et sa configuration propre, peut sembler disproportionné pour extraire une seule fois une petite liste de données, alors qu'un fichier de configuration DataHarvest de quelques lignes suffit largement.

La deuxième situation est plutôt pédagogique. DataHarvest tient dans une poignée de fichiers qu'on peut lire d'un bout à l'autre en une demi-heure, ce qui permet de comprendre précisément ce qui se passe entre le moment où on récupère une page et celui où les données sont enregistrées, sans la complexité interne de Scrapy (planificateur, filtre de doublons, moteur asynchrone) qui reste, pour un débutant, une boîte assez opaque.

#### L'impact du fonctionnement asynchrone sur la vitesse

Pour donner un ordre de grandeur, imaginons qu'on scrape cent pages avec un délai imposé d'une seconde entre chaque requête, et un temps de réponse moyen du serveur d'environ trois dixièmes de seconde. Avec DataHarvest, qui traite les pages une par une, il faudrait environ cent fois cette seconde et demie, soit un peu plus de deux minutes au total. Avec Scrapy, qui peut par exemple envoyer huit requêtes en même temps tout en respectant le même délai par requête, le même travail prendrait environ huit fois moins de temps, soit un peu moins de dix-sept secondes.

Cet écart, de l'ordre d'un facteur sept à huit dans notre exemple, illustre bien l'intérêt du fonctionnement asynchrone dès qu'on doit traiter un grand nombre de pages. Il s'agit toutefois d'un calcul théorique, qui suppose un temps de réponse constant d'une page à l'autre. Une vraie mesure sur nos machines relève de l'extension proposée en fin de sujet, que nous n'avons pas réalisée dans le temps imparti.

## 4. Cadre légal et éthique

### Ce que dit le robots.txt des sites que nous avons scrapés

Avant de scraper un site, il est important de vérifier son fichier robots.txt, qui indique quelles parties du site les robots ne sont pas autorisés à visiter. Nous avons vérifié ce fichier pour chacun de nos cinq sites.

Pour [books.toscrape.com](https://books.toscrape.com) et [quotes.toscrape.com](https://quotes.toscrape.com), il n'existe tout simplement pas de fichier robots.txt : ces deux sites sont conçus spécifiquement pour être scrapés dans un cadre d'apprentissage, donc aucune restriction ne s'applique. Sur [data.gouv.fr](https://data.gouv.fr), le fichier interdit surtout l'accès aux pages d'administration et à certains exports en masse ou pages de recherche avec des paramètres particuliers, mais pas aux pages de fiches de jeux de données que nous consultons. Sur [blogdumoderateur.com](https://blogdumoderateur.com), les restrictions concernent principalement les pages d'administration du site, les flux RSS et les pages de commentaires, pas les pages de listing d'articles que nous utilisons. Enfin, pour Wikipédia, le fichier robots.txt interdit habituellement l'accès aux pages d'action interne (modification, historique) et à certaines pages spéciales, mais nous n'avons pas pu revérifier ce fichier en direct au moment de la rédaction de ce rapport, à cause d'un souci technique avec notre outil de récupération ce jour-là. Ce point sera à revérifier avant le rendu final. Dans tous les cas, les pages que nous ciblons dans nos différentes configurations ne se trouvent pas dans les zones interdites de ces fichiers.

Le tableau suivant résume cette analyse pour les cinq sites.

| Site                                                            | robots.txt                       | Ce qui est interdit                                          | Respecté par notre config         |
|-----------------------------------------------------------------|----------------------------------|--------------------------------------------------------------|-----------------------------------|
| <a href="https://books.toscrape.com">books.toscrape.com</a>     | Absent                           | Rien, le site n'en publie pas                                | Sans objet                        |
| <a href="https://quotes.toscrape.com">quotes.toscrape.com</a>   | Absent                           | Rien                                                         | Sans objet                        |
| <a href="https://fr.wikipedia.org">fr.wikipedia.org</a>         | Présent, non revérifié en direct | Pages d'action et pages spéciales                            | Oui, sous réserve de vérification |
| <a href="https://data.gouv.fr">data.gouv.fr</a>                 | Présent, vérifié                 | Administration, recherches avec paramètres, exports en masse | Oui                               |
| <a href="https://blogdumoderateur.com">blogdumoderateur.com</a> | Présent, vérifié                 | Administration, flux RSS, commentaires                       | Oui                               |

### La question des données personnelles

Les informations que nous récupérons (titres de livres, citations et leur auteur, tableaux encyclopédiques, informations sur des jeux de données publiques, titres d'articles) ne concernent pas, en principe, des personnes physiques dans un cadre privé. Ce sont des informations sur des œuvres, des institutions ou des contenus déjà publiés publiquement. Le seul cas qui pourrait faire un peu débat est le nom des auteurs de citations, par exemple Albert Einstein, mais il s'agit là de



personnalités publiques citées dans le cadre exact pour lequel leurs citations ont été rendues publiques, ce qui limite fortement le risque, sans totalement l'écarter en théorie.

Si on considère malgré tout qu'il s'agit d'une trace de donnée personnelle, la base légale la plus adaptée, au sens de l'article 6 du RGPD, serait l'intérêt légitime, plutôt que le consentement, impossible à recueillir dans le cadre d'un scraping automatisé, ou l'exécution d'un contrat, qui ne s'applique pas ici. Cet intérêt légitime se justifie par le cadre strictement pédagogique du projet, l'absence de tout profilage des personnes concernées, et le fait qu'on ne récupère que les informations strictement nécessaires à la démonstration technique.

### **Comment DataHarvest essaie de rester respectueux des sites qu'il visite**

Plusieurs choix techniques simples permettent à DataHarvest de se comporter correctement vis-à-vis des sites qu'il scrape. Chaque configuration précise un identifiant clair (un User-Agent) qui indique qu'il s'agit de DataHarvest et fournit une adresse de contact, plutôt que de se faire passer pour un navigateur classique. Un délai est respecté entre chaque requête, ce délai étant réglable selon le site visité. En cas d'erreur du serveur, le programme attend progressivement plus longtemps avant de réessayer, plutôt que de renvoyer immédiatement de nouvelles requêtes de façon agressive. Enfin, la base de données utilisée pour un des formats d'enregistrement impose que chaque adresse ne soit enregistrée qu'une seule fois, ce qui évite de retraiter inutilement un même contenu à chaque nouvelle exécution du programme.

## 5. Difficultés et rétrospective

### Difficulté technique principale

La difficulté la plus concrète rencontrée sur ce projet n'est pas apparue en théorie, mais au tout premier vrai test en conditions réelles. En lançant la commande `python -m dataharvest crawl --config configs/blogdumoderateur.yaml` (capture d'écran ci-dessous, prise le 31 juillet 2026), le rapport de session a montré que la totalité des articles étaient rejetés par la validation, avec le même motif répété pour chaque élément : un champ obligatoire vide.

```
(.venv) PS C:\xampp\htdocs\dashboard\Perso\Cours\Web Scraping\j5\openclaw-Maxime-Gurvan> python -m dataharvest crawl --config configs/blogdumoderateur.yaml
[GET https://www.blogdumoderateur.com/]
[200 OK - 0.14s]
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-31T10:32:18+02:00', 'categorie': 'Productivité'}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-31T07:00:40+02:00', 'categorie': 'Marketing digital'}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-30T17:37:21+02:00', 'categorie': 'Social Media'}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-31T11:13:30+02:00', 'categorie': 'Tech'}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-31T10:23:30+02:00', 'categorie': 'Design'}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-30T09:51:43+02:00', 'categorie': 'Gestion d'entreprise'}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-29T17:03:59+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-29T16:56:24+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-29T14:56:58+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-29T11:15:34+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-29T11:14:34+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-24T07:00:18+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-23T14:48:10+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-23T07:00:47+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-22T11:11:49+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-22T09:24:32+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-22T07:00:20+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-28T09:34:53+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-21T07:46:48+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-16T12:40:00+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-16T08:00:18+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-16T07:00:57+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2024-07-04T09:59:07+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2024-06-25T10:56:21+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-06-30T11:13:29+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-06-30T08:00:19+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2024-08-14T11:11:04+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2024-08-14T11:04:54+02:00', 'categorie': ''}
Item rejete, champ obligatoire vide ou absent (titre) : {'titre': '', 'url': '', 'date': '2026-07-15T10:19:16+02:00', 'categorie': ''}
```

*Capture d'écran : tous les articles rejetés car le titre et l'adresse sont vides*

En regardant de plus près, deux informations étaient bien récupérées, la date et parfois la catégorie, alors que le titre et l'adresse de l'article étaient systématiquement vides. Comme ces deux derniers champs reposent sur le même élément de la page (repris de l'exemple fourni dans le sujet), et que le résultat est vide pour absolument tous les articles plutôt que pour une partie d'entre eux seulement, la seule explication cohérente est que cet élément n'existe tout simplement plus dans la page actuelle du site : [blogdumoderateur.com](https://www.blogdumoderateur.com) a visiblement été redesigné depuis la rédaction du sujet, et son ancienne mise en forme des titres d'articles ne correspond plus à ce qu'on trouve aujourd'hui sur le site.

C'est exactement le genre de risque que nous avons déjà anticipé et noté par écrit dans notre fichier de configuration avant même de lancer un test réel, mais l'avoir anticipé n'a pas suffi tant que ce n'était pas vérifié en conditions réelles. La correction demande d'aller inspecter la page actuelle avec les outils de développement du navigateur, pour retrouver le nouvel élément qui contient le titre de chaque article, et de mettre à jour la configuration en conséquence.

## Ce que nous referions différemment

Nous avons écrit les cinq fichiers de configuration d'un seul bloc, avant de tester le moindre scraping réel sur un vrai site. Avec le recul, valider chaque configuration juste après l'avoir écrite, grâce au mode d'essai qui se contente de récupérer et d'afficher les données de la première page sans rien enregistrer, aurait permis de repérer ce genre de décalage bien plus tôt, site par site, plutôt que de découvrir le problème seulement au moment de lancer un scraping complet sur les cinq sites à la fois.

## Ce que nous n'avons pas eu le temps d'implémenter

Trois fonctionnalités sont restées de côté faute de temps : un mécanisme qui limiterait automatiquement le nombre de requêtes envoyées à un même site sur une courte période, un rapport de session au format HTML plus lisible qu'un simple retour dans la console, et un ajustement automatique du délai entre les requêtes selon la vitesse de réponse du site visité. Les trois nous semblent réalisables sans toucher à l'architecture actuelle du projet, puisqu'il suffirait d'ajouter respectivement un nouveau traitement annexe, une nouvelle méthode sur l'Orchestrator, et une petite adaptation de ce même traitement annexe.

## Répartition des tâches

Gurvan s'est occupé de la lecture de la configuration, de l'extraction des données et de la pagination, ainsi que de l'écriture des cinq fichiers de configuration correspondant aux sites choisis. Maxime s'est occupé du téléchargement des pages, des traitements annexes associés (journalisation, nouvelles tentatives en cas d'erreur), de la validation des données et de leur enregistrement, ainsi que de la structure générale de l'Orchestrator. Cette répartition suit le découpage en étapes suggéré par le sujet, et se retrouve dans l'historique de nos commits Git respectifs.

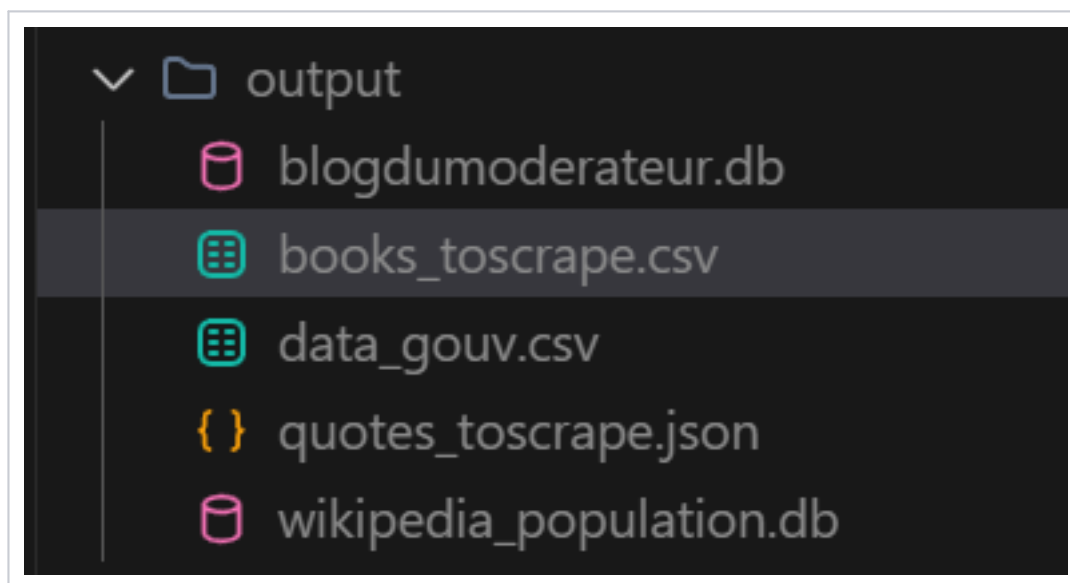
## Preuve : couverture des tests et sortie des cinq sites

Le sujet demande deux choses vérifiables concrètement : une couverture de tests d'au moins 80 %, et un fichier de sortie non vide pour chacun des cinq sites scrapés. Les deux captures ci-dessous, prises le 31 juillet 2026 sur nos machines, servent de preuve pour ces deux points.

La première montre le résultat de la commande `pytest --cov`, qui fait tourner l'ensemble de notre suite de tests en mesurant au passage quelle proportion du code est réellement exécutée par ces tests. Les 57 tests passent tous, et la couverture totale atteint 84 %, ce qui dépasse le seuil de 80 % demandé par le sujet.

```
===== tests coverage =====
----- coverage: platform win32, python 3.14.6-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
dataharvest\__init__.py             1      0   100%
dataharvest\__main__.py             3      3     0%   1-5
dataharvest\app.py                  54     54     0%   1-75
dataharvest\config.py               61      8    87%   19, 22-24, 27, 90, 112, 115
dataharvest\fetcher.py              39      0   100%
dataharvest\middleware.py           35      0   100%
dataharvest\orchestrator.py         38      0   100%
dataharvest\pipeline.py             83      1    99%   108
dataharvest\store.py                73      0   100%
dataharvest\validator.py            32      0   100%
-----
TOTAL                               419     66    84%
===== 57 passed in 3.86s =====
```

La seconde montre le contenu du dossier `output/` après avoir lancé DataHarvest sur nos cinq configurations. On y retrouve bien un fichier par site, dans le format choisi pour chacun : `books_toscraper.csv`, `quotes_toscraper.json`, `wikipedia_population.db`, `data_gouv.csv` et `blogdumoderateur.db`. Chacun de ces fichiers contient des données, ce qui confirme que les cinq configurations produisent une sortie exploitable, y compris `blogdumoderateur.yaml`, une fois le sélecteur corrigé suite au bug décrit plus haut.



Capture d'écran : dossier output avec les cinq fichiers générés, un par site

## 6. Perspectives

---

### Une sixième brique possible : un système de notification

Une brique supplémentaire intéressante serait un système qui préviendrait automatiquement la personne à l'origine du scraping, une fois celui-ci terminé, en lui envoyant le rapport de session déjà produit par le programme (par email, par message sur un outil comme Slack, ou par un simple appel à une adresse web). Comme les autres briques du projet, elle s'intégrerait en étant fournie de l'extérieur plutôt qu'importée directement dans le code, ce qui la rendrait facile à tester séparément.

```
from abc import ABC, abstractmethod

class BaseNotifieur(ABC):
    @abstractmethod
    def notify(self, report: dict) -> None:
        """Transmet le rapport de session via un canal donne."""

class WebhookNotifieur(BaseNotifieur):
    def __init__(self, webhook_url: str):
        self.webhook_url = webhook_url

    def notify(self, report: dict) -> None:
        requests.post(self.webhook_url, json=report, timeout=5)
```

### Comment gérer les sites en JavaScript sans passer par Selenium

Deux pistes plus légères que Selenium existent pour gérer les sites dont le contenu n'apparaît qu'après exécution de JavaScript. La première est Playwright, dont le fonctionnement est nativement pensé pour l'asynchrone et qui démarre plus rapidement que Selenium, tout en pouvant être utilisé simplement pour récupérer le HTML final d'une page avant de le repasser au reste du programme sans rien changer d'autre. La seconde est de placer, devant notre système de téléchargement actuel, un service de pré-rendu comme Splash, qui se charge d'exécuter le JavaScript à notre place et nous renvoie directement le HTML déjà prêt à être analysé, exactement comme si la page n'avait jamais eu besoin de JavaScript.

### Ce qu'il faudrait ajouter pour distribuer DataHarvest sur PyPI

Pour publier DataHarvest en tant que vrai paquet Python installable par tout le monde, il faudrait au minimum ajouter un fichier de métadonnées de projet, adopter une vraie politique de numérotation des versions au-delà de la version actuelle, tester que le paquet s'installe correctement une fois construit, choisir et afficher clairement une licence d'utilisation, écrire une documentation destinée aux utilisateurs plutôt qu'aux seuls développeurs du projet, et mettre en place une intégration continue qui publie automatiquement une nouvelle version à chaque nouvelle étiquette posée sur le dépôt Git.

## Bibliographie et sources

---

Toutes les adresses ci-dessous ont été revérifiées le 31 juillet 2026 pour confirmer qu'elles pointent bien vers les bonnes pages.

| Ressource                                            | Lien                                                                                                                                    |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Documentation Scrapy                                 | <a href="https://docs.scrapy.org">https://docs.scrapy.org</a>                                                                           |
| Documentation Beautiful Soup 4                       | <a href="https://www.crummy.com/software/BeautifulSoup/bs4/doc/">https://www.crummy.com/software/BeautifulSoup/bs4/doc/</a>             |
| Documentation Requests                               | <a href="https://requests.readthedocs.io">https://requests.readthedocs.io</a>                                                           |
| Documentation PyYAML                                 | <a href="https://pyyaml.org/wiki/PyYAMLDocumentation">https://pyyaml.org/wiki/PyYAMLDocumentation</a>                                   |
| Documentation pytest                                 | <a href="https://docs.pytest.org">https://docs.pytest.org</a>                                                                           |
| RGPD, article 6 (bases legales du traitement) - CNIL | <a href="https://www.cnil.fr/fr/reglement-europeen-protection-donnees">https://www.cnil.fr/fr/reglement-europeen-protection-donnees</a> |
| RFC 9309, Robots Exclusion Protocol                  | <a href="https://www.rfc-editor.org/rfc/rfc9309">https://www.rfc-editor.org/rfc/rfc9309</a>                                             |

Les fichiers robots.txt de data.gouv.fr et blogdumoderateur.com ont également été consultés directement le 31 juillet 2026, en complément de cette liste.